

City Water Supply Forecasting

IBM Internship Project — Problem 010 · Reservoir level forecasting dashboard built on Indian government open data (data.gov.in / CWC / IMD)

Version 1.0

Prepared: Sep 2026

Models: Linear Regression + Random Forest

Deliverable: Public dashboard + GitHub repo

1. Context
2. Users
3. Scope
4. Requirements
5. Data
6. Architecture
7. Success Metrics
8. Timeline
9. Gaps to Cover
10. Existing Applications
11. Recommendations

1. Context & Problem Statement

Indian cities repeatedly face acute water crises — most visibly Chennai in 2019, where all four major reservoirs ran dry. Reservoir management, rationing decisions, and public communication currently rely on manual weekly bulletins rather than forward-looking, publicly accessible forecasts.

Objective: build a system that predicts reservoir water levels using historical rainfall, usage, and reservoir data sourced from government open-data portals (data.gov.in, CWC, IMD), and expose it through a public dashboard.

Definition of success (client-stated): a municipal official can spot a potential shortage weeks in advance; a citizen can understand current water status at a glance.

2. Users & Personas

Municipal / water-board official

Needs early warning (weeks, not days), confidence in the number, and a comparison against historical norms to justify rationing decisions.

Citizen / resident

Needs a one-glance status (green/yellow/red), plain-language explanation, no technical jargon, works on a low-end phone.

Journalist / researcher

Wants historical trends, downloadable data, and transparent methodology to cite responsibly.

Reviewer (IBM mentor / evaluator)

Wants to see clean engineering practice: reproducibility, documented data sourcing, honest model evaluation.

3. Scope

In scope (MVP — compulsory)	Stretch (good-to-have)	Out of scope
• 1 (or a small set of) reservoir(s), one region • Cleaned, merged rainfall + reservoir dataset • Linear Regression & Random Forest models • Deployed public dashboard (current level + forecast) • Documented GitHub repo • Final presentation on data challenges	• Multi-reservoir comparison view • Plain-language citizen alerts • Historical crisis case study (2019 Chennai) • Year-over-year comparison ("30% below 2019 average")	• Real-time IoT sensor integration • Water quality / potability modeling • Automated rationing / control actions • Mobile native app

4. Functional & Non-Functional Requirements

Functional

ID	Requirement
FR-1	Ingest daily/weekly reservoir level data from data.gov.in / CWC for the chosen reservoir(s).
FR-2	Ingest matching rainfall data (IMD / data.gov.in / Kaggle India Rainfall) for the same region and time window.
FR-3	Clean, standardize units (mm vs cm, MCM vs TMC), align dates, and merge into one time series, with documented handling of missing values.
FR-4	Train and compare a Linear Regression and a Random Forest model to forecast reservoir level over a defined horizon (e.g., 7–30 days).
FR-5	Report honest model performance (MAE, RMSE, R^2) using a time-based train/test split, not a random split.
FR-6	Dashboard: current level, forecast curve, historical comparison, seasonal chart.
FR-7	Visual concern indicator (green/yellow/red) with a stated, documented threshold logic.
FR-8	Descriptive analytics view: seasonal rainfall/level patterns, at least one identified pattern called out explicitly.
FR-9	Public GitHub repo: data sourcing steps, cleaning decisions, model code, README, reproducible pipeline.

Non-functional

- **Usability:** the citizen view must be understandable with zero domain knowledge — a color and one sentence, not a chart full of axes.
- **Transparency:** every number on the public dashboard should be traceable to a documented source and date of last update.
- **Performance:** dashboard loads in a few seconds on a typical mobile connection.
- **Maintainability:** pipeline should be re-runnable when new government data drops (weekly bulletins), not a one-off notebook.

- **Honesty about uncertainty:** forecasts should show a confidence range, not a single deceptively precise number.

5. Data Sources

Need	Source options
Reservoir level / storage	data.gov.in (CWC daily reservoir data), CWC Reservoir Storage Bulletin, India-WRIS, state water board portals (e.g., CMWSSB for Chennai)
Rainfall	data.gov.in, IMD historical archives, Kaggle "India Rainfall" datasets, Open-Meteo (for gap-filling/validation)
Usage / demand context (optional)	City water-board consumer billing reports, population growth data, if publicly available

6. Suggested Architecture

1. **Ingestion:** scripted download from data.gov.in API / CSV exports + IMD archives.
2. **Cleaning layer:** pandas scripts — unit conversion, date parsing, interpolation for short gaps, explicit flags for long gaps (never silently fill with zero).
3. **Feature store:** merged table keyed by date + reservoir, with lag features (rainfall lag 7/14/30 days, seasonal index).
4. **Modeling:** scikit-learn Linear Regression + Random Forest, versioned with a simple experiment log (params, metrics, date).
5. **Serving:** Streamlit (fastest path) or Dash/Gradio, reading the latest model artifact + latest data snapshot.
6. **Docs:** README with data-sourcing narrative, a CHANGELOG, and a model card describing limitations.

7. Success Metrics

- Forecast error (RMSE/MAE) beats a naive baseline (persistence / seasonal-average model) — this is the real bar, not an absolute accuracy number.
- Dashboard shows current status + at least a 30-day outlook.
- At least one genuine, documented seasonal pattern surfaced from the data.
- Repo documentation lets a stranger reproduce the pipeline end-to-end.
- A non-technical reviewer can state the current water status after 10 seconds on the dashboard.

8. Suggested Timeline

Phase	Focus
Week 1	Source & download reservoir + rainfall data; document every dead end (this is a graded skill, not a side effect).

Week 2	Clean, convert units, merge, handle missing years; produce descriptive/seasonal analysis.
Week 3	Train Linear Regression + Random Forest with a time-based split; evaluate against a naive baseline.
Week 4	Build and deploy dashboard; add concern-level coloring; write documentation.
Week 5	Stretch goals (multi-reservoir, alerts, year-over-year comparison); polish final presentation.

9. Gaps You Need to Cover

These go beyond what the problem statement spells out — they're the things reviewers (and real deployments) get judged on.

- 1 Time-based validation, not random split.** The brief flags this, but it's worth internalizing why: random splits leak future information into training via seasonal autocorrelation, making the model look far better than it will be in production.
- 2 Baseline comparison.** A model isn't "good" in isolation — compare against a naive baseline (last value repeated, or same-month historical average). If Random Forest barely beats a naive baseline, that's an important, honest finding to report.
- 3 Uncertainty communication.** A single forecasted number invites false confidence. Show a range or confidence band, and be explicit about forecast horizon limits (a 7-day forecast is much more trustworthy than a 90-day one).
- 4 Data provenance & licensing.** Document exactly which dataset version, download date, and license (Open Government License, India) each file came from — critical for reproducibility and for citing in the GitHub repo.
- 5 Missing-data strategy documentation.** Don't just interpolate — record how many rows/years were affected, why, and what interpolation method was chosen and why it's defensible for hydrological data (which has strong seasonality).
- 6 Drift & retraining plan.** Government bulletins update weekly. Design the pipeline (even conceptually) for periodic retraining, not a one-time model that decays silently.
- 7 Accessibility & language.** If citizen usage is a real goal, consider a regional-language toggle and colorblind-safe status indicators, not just green/yellow/red hues.
- 8 Explainability.** For a municipal official, "why is the model predicting a drop" matters more than the number itself. Feature importance from Random Forest (rainfall lag, seasonal index) should be surfaced, not buried in the notebook.
- 9 Threshold logic for alerts.** "Green/yellow/red" needs a defined, defensible rule (e.g., percentile of historical storage for that calendar week) — arbitrary thresholds undermine trust the moment someone asks "why is it red?"
- 10 Hosting & cost reality.** Streamlit Community Cloud / Hugging Face Spaces are free but sleep on inactivity and have limited compute — fine for a demo, worth flagging as a limitation if pitched as "production-ready."

11 Attribution and disclaimers. Since this could be mistaken for an official source, the dashboard should clearly state it's an independent/academic project, not an official government tool, with a link back to the source bulletins.

12 Cross-validation of the "crisis narrative." If you cite the 2019 Chennai crisis as context or in a bonus year-over-year feature, ground it in the actual CWC/CMWSSB historical bulletins rather than approximate figures from memory.

10. Landscape — 10 Existing Applications on This Problem

A mix of official Indian government systems, global AI-driven platforms, a US analog, and open-source/academic projects closest to the internship's own scope — useful both as inspiration and as a bar to differentiate against.

1. CWC Reservoir Storage Bulletin / Web-based Reservoir Storage Monitoring System

Govt of India · Official

Central Water Commission's weekly bulletin (issued every Thursday) covering live storage of 91+ major reservoirs nationwide, compared against last year and the 10-year normal. Relaunches as a web portal in 2025.

PROS

- Authoritative, nationwide coverage
- Long historical baseline (10-yr average)
- Free, open government data

CONS

- Weekly cadence, not real-time
- No predictive forecasting, only current/historical status
- Dense, technical presentation — not citizen-friendly

2. India-WRIS WebGIS

CWC + NRSC/ISRO · Official

National "single window" GIS platform integrating watershed, reservoir, telemetry, and thematic water-resource layers across India, jointly built by CWC and ISRO's NRSC.

PROS

- Extremely rich, multi-layer GIS dataset
- Live telemetry + reservoir module
- Custom map/report generation tools

CONS

- Steep learning curve for non-experts
- No forward-looking ML forecasts
- Interface not designed for a general citizen audience

3. CWC FloodWatch India App 2.0

Govt of India · Mobile app

CWC's mobile app for flood-related forecasts and advisories, extending the Commission's flood forecasting network to a citizen-facing app.

PROS

- Mobile-first, reaches citizens directly
- Backed by CWC's official gauge network

CONS

- Focused on flood (excess water), not scarcity/reservoir depletion forecasting
- Coverage concentrated on major flood-prone river systems

4. Chennai Metro Water (CMWSSB) Lake Level Portal & App

Tamil Nadu Govt · Regional

CMWSSB publishes daily lake/reservoir storage, rainfall, and groundwater-level details for Chennai's supply reservoirs (Poondi, Cholavaram, Red Hills, Chembarambakkam), plus a citizen-facing "Chennai Kudineer" app for billing/complaints.

PROS

- Hyper-local, exactly the crisis city referenced in the brief
- Combines rainfall + lake + groundwater status

CONS

- Purely descriptive — current/historical figures, no predictive model
- Utilitarian interface, low visual/civic-communication value

5. Google Flood Hub

Google Research · Global AI platform

AI-powered riverine and urban flash-flood forecasting using LSTM-based global hydrology models, satellite imagery, and gauge data — piloted in the Ganges-Brahmaputra basin (India) in 2018, now covering 80+ countries with up to 7-day lead time, surfaced in Search/Maps.

PROS

- State-of-the-art deep learning, works even with sparse local data
- Massive scale, proven humanitarian impact (documented reduction in flood-related medical costs in Bihar)
- Free, public, integrated into products people already use

CONS

- Built for flood risk, not reservoir/supply-side scarcity forecasting
- Accuracy varies where gauge data is sparse
- Closed model internals — not a template a student project can replicate directly

6. USBR RISE — Reservoir Conditions Dashboard

US Bureau of Reclamation · Government (US analog)

Interactive "teacup diagram" dashboard showing current storage at 54 US Reclamation reservoirs against a 30-year historical average, with downloadable CSV data back to 1990.

PROS

- Excellent, simple visual metaphor (teacup fill level) for non-experts
- Long clean historical record, open data/API access

CONS

- Descriptive/historical comparison only, not predictive
- US-specific; not directly usable for Indian datasets/units

7. USGS National Water Dashboard / WaterWatch

USGS · Government (US analog)

Real-time map of 13,500+ US streamflow, lake, reservoir, and groundwater stations with percentile-based status coloring and an alert-subscription system (WaterAlert).

PROS

- Percentile-based status is a good, defensible model for the brief's "green/yellow/red" requirement
- Push alerts via email/SMS — a strong pattern for the citizen-alert stretch goal

CONS

- Dense, GIS-heavy interface; not built for a quick public glance
- No forward forecast — only real-time + historical percentile

8. EFAS / GloFAS (Copernicus Emergency Management Service)

EU / ECMWF · Global government platform

Europe-wide and global operational hydrological forecasting (LISFLOOD model + ECMWF ensemble weather) providing river-discharge and flood-probability forecasts up to 15–30 days ahead.

PROS

- Rigorous ensemble-forecast methodology with explicit skill/uncertainty reporting
- Long operational track record (since 2012), strong institutional backing

CONS

- Real-time EFAS data restricted to registered hydrological/civil-protection authorities, not fully public
- Built for river discharge/flood risk, not reservoir supply management

9. "neer-vazhvu" — Urban Water Intelligence Dashboard

Open-source / independent project

An open-source dashboard covering Chennai and Bengaluru water bodies: reservoir forecasting, groundwater-risk scoring, pollution-source overlays, and AI-generated daily briefings, built on IMD, CGWB, India-WRIS, and satellite (Earth Engine) data.

PROS

- Closest real-world analog to this exact brief — multi-source govt-data integration for the same cities
- Goes beyond reservoir level into groundwater risk and plain-language briefings (a good model for the "citizen-alert layer" stretch goal)

CONS

- Independent/community project — not an official or continuously funded service
- Heavier engineering footprint (GIS zonal stats, LLM narratives) than a typical internship scope

10. Student / academic Streamlit forecasting projects

GitHub · e.g. "water-supply-forecasting", "Rainfall-Forecaster"

A cluster of similar public GitHub projects (including one using near-identical framing to this brief — comparing Linear Regression and Random Forest on Indian reservoir/rainfall data, with an optional LSTM extension) and IMD-based rainfall forecasting Streamlit apps using Random Forest / Auto-ARIMA.

PROS

- Directly comparable scope — good for benchmarking your own approach and avoiding redundant mistakes

- Streamlit-based, quick to deploy, minimal infra needed

CONS

- Typically single-region, small-dataset prototypes — not built for reliability or citizen trust at scale
- Documentation of data-cleaning decisions is often thin, which is precisely the skill this internship grades on

11. How to Differentiate Your Project

- Nobody in this landscape combines **predictive forecasting** (Google Flood Hub, EFAS/GloFAS) with **reservoir-specific supply framing** (CWC, CMWSSB, USBR) *and* a genuinely citizen-friendly interface. That gap is your opening.
- Borrow the **percentile-based status coloring** from USGS WaterWatch and the **teacup visual metaphor** from USBR RISE — both are more defensible and intuitive than an arbitrary threshold.
- Borrow the **alert-subscription pattern** from USGS WaterAlert for the "citizen-alert layer" good-to-have.
- Explicitly document your data-cleaning process better than the student projects reviewed above — the brief calls this out as "a key demonstrated skill," and it's the most common weak point across comparable projects.
- Be candid in your final presentation about what your model *can't* do (short lead time, single-region, sparse years) — this maturity is what separates a strong internship deliverable from a toy notebook.

Prepared as a working PRD for an IBM internship deliverable · Sources: data.gov.in, CWC, CMWSSB, Google Research, Copernicus/ECMWF, USBR, USGS, public GitHub repositories.